

Introduction to Machine Learning (SS 2026)

Programming Project

Author 1

Last name: [TODO: last name]

First name: [TODO: first name]

Matrikel Nr.: [TODO: Matrikel Nr.]

Author 2

Last name: Beneduce

First name: Paul

Matrikel Nr.: 12309055

I. INTRODUCTION

This is a supervised *regression* task: we predict the daily total `power_consumption` of a region from weather data. This is useful for energy suppliers, since they need to match how much power they generate with how much is actually used.

The dataset (`powerpredict`) has 39,997 rows and 65 input features. Each feature describes the weather in one of five fictional cities (Bedrock, Gotham City, New New York, Paperopolis, Springfield). For each city there are 11 numeric measurements (temperature and its daily low/high, pressure, humidity, wind speed and direction, short-/long-term rain, snow, cloud cover) and 2 text columns (`weather_main`, `weather_description`), which gives 55 numeric and 10 categorical columns in total. The target is one continuous value. There are no missing values in the numeric features, so we did not need to fill anything in, but the features have very different scales (e.g. pressure vs. rainfall), which is why we scale them below. The target ranges from 18,041 to 41,015, with a mean of 28,555 and a standard deviation of 4,735. Its distribution is roughly symmetric with one main peak, but there is also a small, narrow spike around 23,000. We evaluate all models with the *Mean Absolute Error* (MAE), since this is the official metric of the competition (lower is better).

II. IMPLEMENTATION / ML PROCESS

Data preprocessing

Both methods use the same preprocessing step, built with a scikit-learn `ColumnTransformer` inside a `Pipeline`. Numeric columns are scaled with `StandardScaler` (zero mean, unit variance). This step matters a lot for a distance-based method like k-NN: without it, a feature with a large range (e.g. pressure) would dominate the distance calculation and features with a small range (e.g. rainfall) would barely matter. The text weather columns are turned into 0/1 columns with `OneHotEncoder` (`handle_unknown="ignore"`, so weather categories that only show up in the hidden test set do not cause an error). We decided to test both with and without these categorical columns, and compare the two

options in Section III. Putting the scaler, encoder and model together in one pipeline makes sure the scaler and encoder are only fitted on the training folds, so there is no data leakage during cross-validation, and it also lets the evaluation notebook use the raw table directly.

Method 1: k-Nearest Neighbors regression

Method 1 is a k-Nearest-Neighbors (k-NN) regressor. It is an instance-based method that does not learn any parameters: it just stores the (scaled) training data and predicts a new point by averaging the targets of its k closest neighbours, using the Euclidean (Minkowski, $p = 2$) distance. We use distance weighting, so neighbours that are closer have a bigger influence on the prediction than neighbours that are far away.

k-NN fits this problem well because the relationship between weather and power consumption is probably non-linear but locally smooth: days with similar weather should have similar power consumption. This is exactly what k-NN uses, without assuming any fixed global formula. It is also a simple but strong contrast to the linear Method 2. Since it does not learn parameters, k-NN is closely related to the non-parametric density estimation from the VO [1]; k-NN regression basically replaces the density estimate with a local average of the target values.

Hyperparameters

We chose the hyperparameters with a full grid search using cross-validation, scored by (negative) MAE, so the choice is based on held-out folds and not on the training data itself. The grid was the number of neighbours $k \in \{3, 5, 11, 25, 51\}$ and the weighting method (*uniform* or *distance*). The best combination was $k = 3$ with distance weighting (CV MAE 1799.14). The final submitted model only uses the numeric features (categorical encoding turned off): dropping the one-hot columns kept the memory usage inside the JupyterHub limit, and it did not really hurt accuracy (see Section III).

III. RESULTS

Table I shows the cross-validated MAE of the k-NN model for every hyperparameter combination, and Fig. 1 shows

how the validation MAE changes with k . A small k with distance weighting gave the lowest error, so we selected $k = 3$. Table II shows the final performance compared to the dummy baseline (a model that always predicts the mean of the training set).

TABLE I
CROSS-VALIDATED MAE OF THE K-NN MODEL FOR EACH
HYPERPARAMETER CONFIGURATION (LOWER IS BETTER).

k (neighbours)	weights	CV MAE
3	uniform	2021.10
3	distance	1799.14
5	uniform	2174.64
5	distance	1838.56
11	uniform	2414.77
11	distance	1916.81
25	uniform	2655.23
25	distance	2012.32
51	uniform	2816.70
51	distance	2075.85

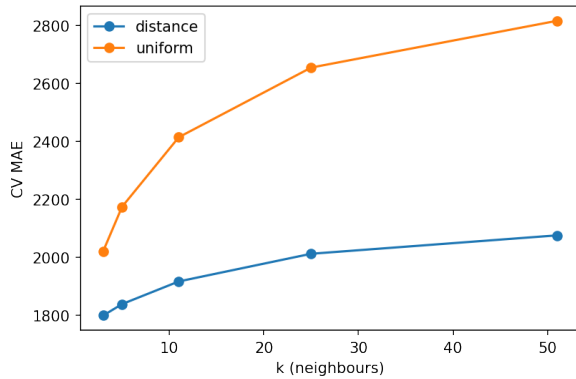


Fig. 1. Validation MAE of the k-NN regressor as a function of the number of neighbours k , for uniform and distance weighting.

TABLE II
FINAL MAE ON THE TRAINING, VALIDATION AND (HIDDEN) TEST SETS.
LOWER IS BETTER.

Model	Train MAE	Valid. MAE	Test MAE
Dummy (mean)	4086.00	4080.71	3932.85
k-NN (Method 1)	17.97	1799.14	1835.41
Ridge (Method 2)	[TODO:]	[TODO:]	[TODO:]

IV. DISCUSSION

The k-NN model clearly beats the dummy baseline on unseen data (test MAE 1835.41 vs. dummy 3932.85), which shows that the weather features really do carry useful information. The very low *training* MAE (17.97) is expected and does not mean the model is good: with distance weighting, if a query point is the same as a training point, its weight goes to infinity, so the model almost perfectly reproduces the training data. The gap to the validation/test MAE is the real generalisation error, and this is the number that actually matters.

The test MAE (1835.41) is a bit higher than the cross-validation estimate (1799.14), which is what we would expect: the final model is trained again on the full training set, so this small gap is just normal generalisation error, not a mistake in how we evaluated it. Since the gap is small, the model does not seem to be overfitting on the public data.

We tried two things that did not work out: (i) One-hot encoding the weather-text columns made the feature space much bigger, and together with how much memory k-NN needs, this used up all the RAM on JupyterHub (the training process got killed). On a smaller subsample of $n=8000$ rows (used because the full run crashed), the accuracy gain was basically zero, and even slightly worse (val. MAE 2532.75 with encoding vs. 2388.89 without), so we kept the simpler numeric-only model. (ii) Running the grid search in parallel (`n_jobs=-1`) copied the whole dataset into every worker process, which caused the same out-of-memory crash, so we ran the search step by step instead.

The best k (3) was at the lower end of our grid, which suggests that an even smaller neighbourhood could reduce the error even more; trying smaller values (e.g. $k \in \{1, 2\}$) would be the obvious next step. Beyond that, dimensionality reduction such as PCA [2] could help with the curse of dimensionality, which is a problem for distance-based methods, and feature selection or a different distance metric could also improve the neighbour search.

V. CONCLUSION

Our final k-NN regressor reaches a test-set MAE of **1835.41**, which is much better than the dummy baseline of 3932.85. We did not tune anything on the test set; the hyperparameters were only chosen based on cross-validated performance on the training data.

The main takeaway is that a simple, non-parametric method with careful preprocessing already gets you quite far on this problem: scaling the weather features and letting k-NN average over similar days gives a big improvement over the mean predictor. This project also showed the practical downsides of instance-based learning: k-NN's memory cost forced us to drop the categorical features and run the grid search step by step, and the fact that it memorises the training data makes the training error misleading. A regularised linear model (Method 2) gives a different, more memory-friendly perspective, and comparing the two shows when local averaging works better than a global linear fit for predicting power demand from weather.

REFERENCES

- [1] Introduction to Machine Learning (SS 2026), Lecture 10: *Nonparametric Density Estimation*. University of Innsbruck.
- [2] Introduction to Machine Learning (SS 2026), Lecture 07: *Dimensionality Reduction by PCA/SVD*. University of Innsbruck.